

架构体检报告

时间剪史

2026-06-07 · SwiftUI + AppKit macOS 剪贴板历史工具

☐ 实线框 = 模块

----- 虚线 = 接缝

→ 红色箭头 = 泄漏

☒ 深色框 = 更深的模块

把“剪贴板摄取”做成更深的模块

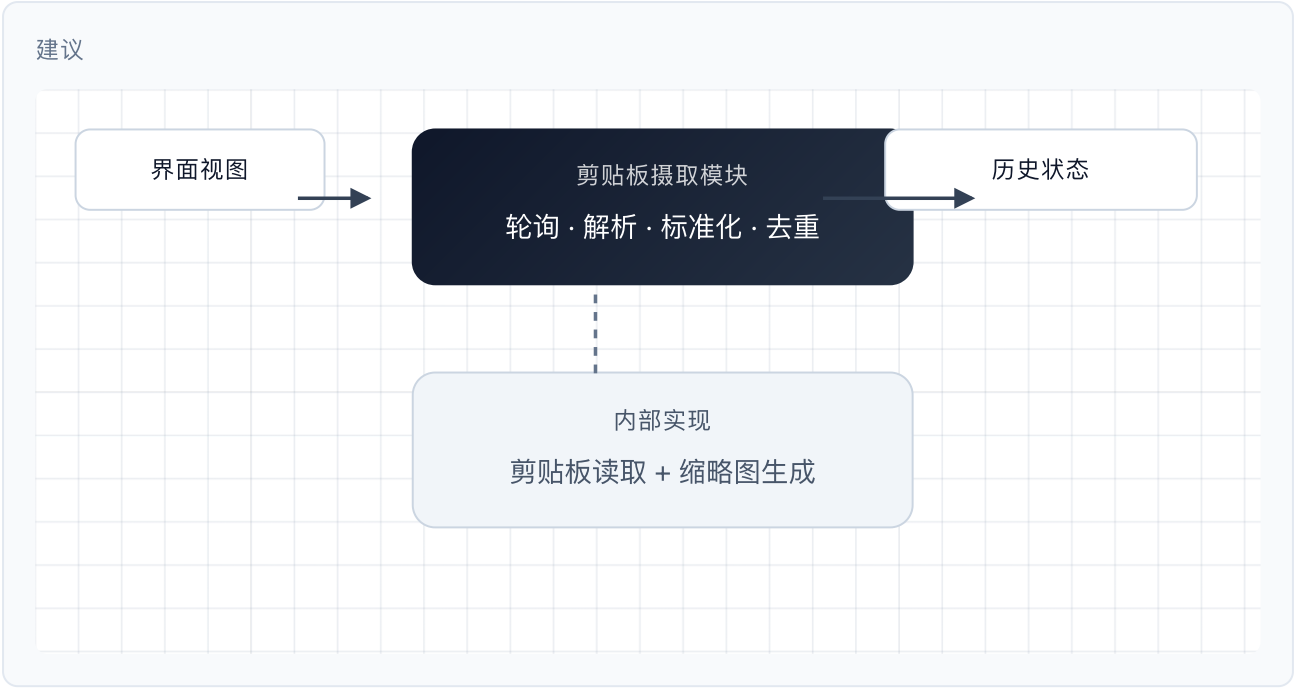
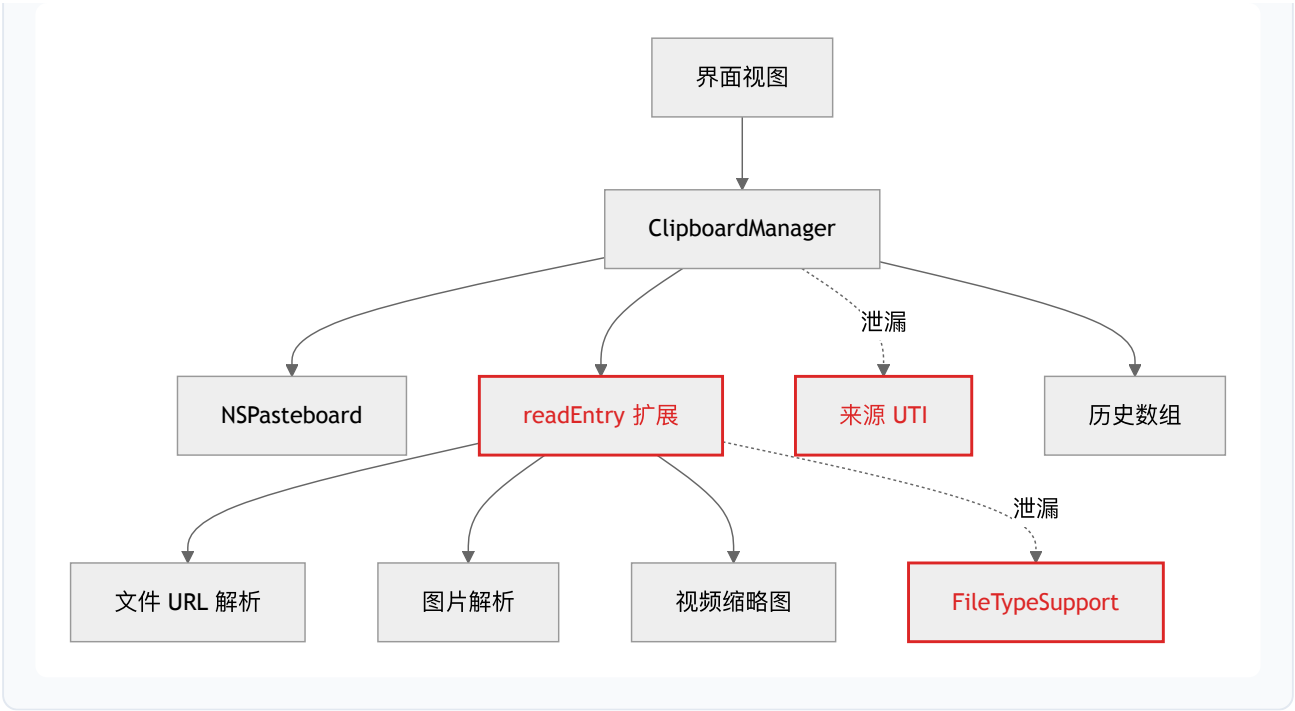
强烈建议

进程内模块

涉及文件

```
ClipboardHistory/Sources/ClipboardHistoryApp/Managers/ClipboardManager.swift
ClipboardHistory/Sources/ClipboardHistoryApp/Managers/ClipboardReader.swift
ClipboardHistory/Sources/ClipboardHistoryApp/Models/ClipboardEntry.swift
ClipboardHistory/Sources/ClipboardHistoryApp/Models/EntryContent.swift
```

现在



问题

ClipboardManager 的接口偏浅：调用者会同时碰到历史状态、剪贴板时序、解析顺序和缩略图副作用。

建议

做一个剪贴板摄取模块：对外只交付标准化条目，对内隐藏剪贴板解析和缩略图实现。

收益

- 解析问题集中处理
- 一个摄取接口覆盖多场景
- 测试只看标准化条目

- 缩略图成本变明确

把“文件预览”做成一个模块

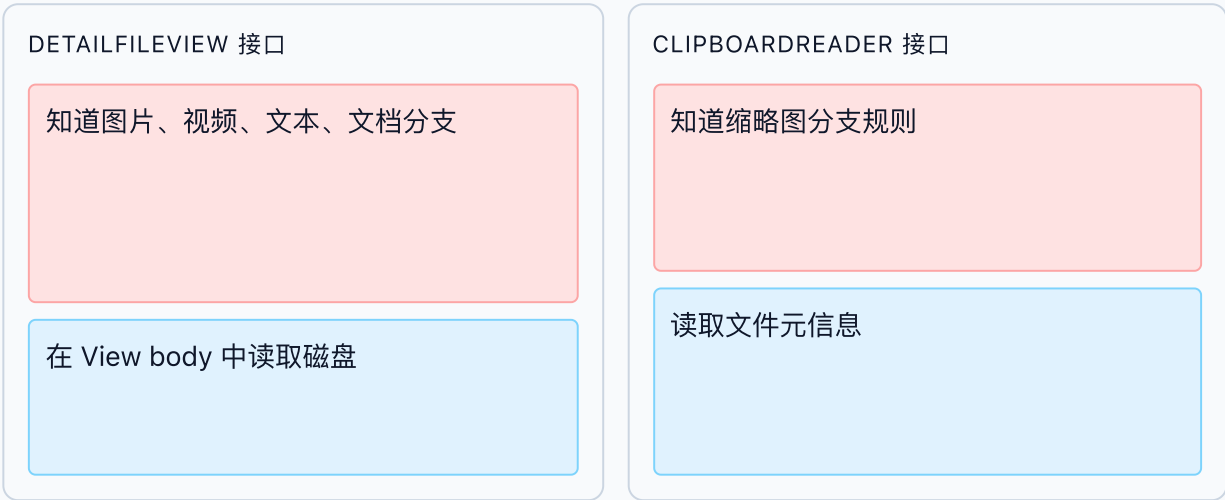
强烈建议

本地可替换

涉及文件

```
ClipboardHistory/Sources/ClipboardHistoryApp/Views/DetailPreviewViews.swift
ClipboardHistory/Sources/ClipboardHistoryApp/Managers/ClipboardReader.swift
ClipboardHistory/Sources/ClipboardHistoryApp/Utilities/FileTypeSupport.swift
ClipboardHistory/Sources/ClipboardHistoryApp/Views/ImagePreviewView.swift
```

现在



文件类型判断泄漏在多个接缝之间。

建议

文件预览模块

接口：根据 URL + 可选缩略图返回预览状态

实现：图片读取、文本读取、QuickLook 选择、fallback

内部接缝：大文件异步加载

问题

DetailFileView 表面上只是 View，但内部在做文件分类和磁盘 IO，接口偏浅。

建议

把文件预览选择和加载放到一个模块里，View 只负责渲染预览状态。

收益

- 文件规则集中
- 一个预览接口
- 测试不用跑 SwiftUI body
- 异步加载有接缝

把“应用外壳”做深

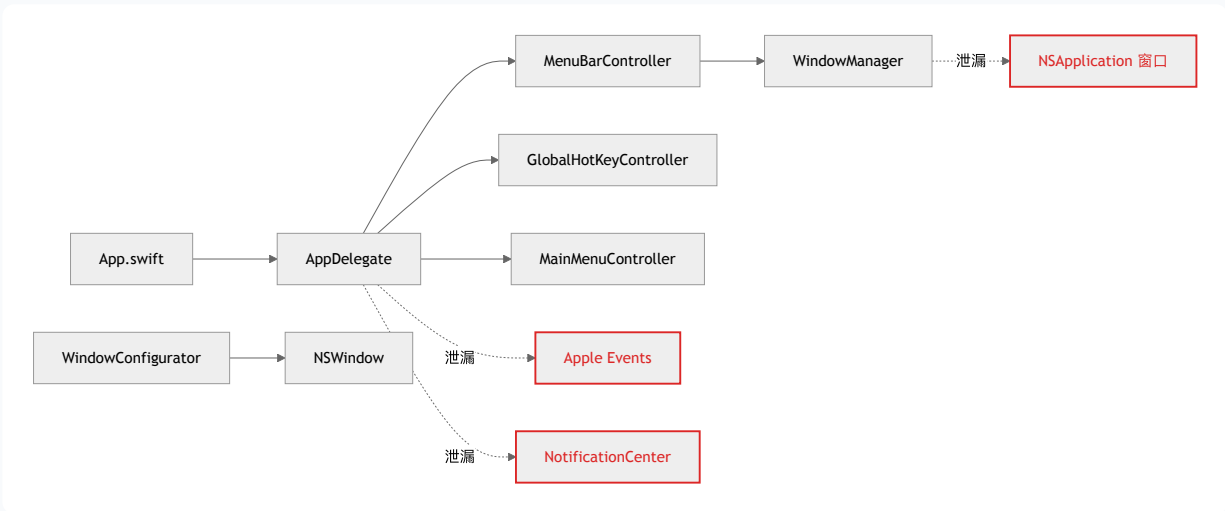
值得探索

端口与适配器

涉及文件

```
ClipboardHistory/Sources/ClipboardHistoryApp/App.swift
ClipboardHistory/Sources/ClipboardHistoryApp/Managers/AppDelegate.swift
ClipboardHistory/Sources/ClipboardHistoryApp/Managers/MenuBarController.swift
ClipboardHistory/Sources/ClipboardHistoryApp/Managers/WindowManager.swift
ClipboardHistory/Sources/ClipboardHistoryApp/Managers/WindowConfigurator.swift
```

现在



建议

应用外壳模块

显示主窗口 · 关闭时隐藏 · Dock 重开 · 退出 · 设置

macOS 12 NSStatusItem macOS 13 MenuBarExtra 适配器

NSWindow 适配器

问题

窗口生命周期知识分散在多个模块里，macOS 12/13 行为很难通过一个接口验证。

建议

创建应用外壳模块：接口只描述用户意图，AppKit 细节藏在适配器后面。

收益

- 生命周期问题集中
- 一个意图接口
- 适配器接缝合理
- 回归测试可按意图脚本化

把“快捷键配置”做深

值得探索

本地可替换

涉及文件

ClipboardHistory/Sources/ClipboardHistoryApp/Models/HotKeyShortcut.swift
ClipboardHistory/Sources/ClipboardHistoryApp/Utilities/HotKeyPreferences.swift
ClipboardHistory/Sources/ClipboardHistoryApp/Managers/GlobalHotKeyController.swift
ClipboardHistory/Sources/ClipboardHistoryApp/Views/SettingsView.swift
ClipboardHistory/Sources/ClipboardHistoryApp/Views/HotKeyRecorderView.swift

现在

- HotKeyShortcut：校验 + 展示 + Carbon 键名
- HotKeyPreferences：UserDefaults 编解码
- SettingsView：保存 + 失败回滚
- GlobalHotKeyController：Carbon 注册

NotificationCenter：变更与失败消息

建议

快捷键设置模块

接口：当前值、录制、保存、重置、注册状态

实现：校验、持久化、Carbon 适配器、失败回滚

问题

快捷键变更流程分散在模型、View、偏好设置、通知和 Carbon 注册里，接口偏浅。

建议

让快捷键配置成为一个模块；Carbon 只是适配器，注册状态直接通过接口暴露。

收益

- 失败回滚只写一次
- 一个设置接口
- 测试可替换 Carbon 适配器
- 通知不再泄漏

把“展示文案格式化”收进小模块

暂时观察

进程内模块

涉及文件

ClipboardHistory/Sources/ClipboardHistoryApp/Models/EntryContent.swift
ClipboardHistory/Sources/ClipboardHistoryApp/Models/ClipboardEntry.swift
ClipboardHistory/Sources/ClipboardHistoryApp/Views/DetailView.swift
ClipboardHistory/Sources/ClipboardHistoryApp/Views/HistoryRowViews.swift
ClipboardHistory/Sources/ClipboardHistoryApp/Utilities/ClipboardDateFormatters.swift

现在

EntryContent
预览文案

DetailView
尺寸文案

HistoryRow
文件名拆分

DateFormatters

FileTypeSupport

README 术语

建议

条目展示模块

小接口：行标题、详情副标题、尺寸文案

较大的实现：本地化、截断、文件名、日期、类型描述

问题

展示字符串散在模型和 View 里；现在不严重，但本地化继续增长后会变浅。

建议

等 v1.3 文案和本地化增多时，把预览、尺寸、文件名展示收进一个格式化模块。

收益

- 文案变化集中
- 一个格式化接口
- 测试不用渲染 SwiftUI
- 模型更纯粹

最推荐先做

先从 把“剪贴板摄取”做成更深的模块 开始。

它最靠近“时间剪史”的核心价值，也会直接帮助后续历史持久化、缩略图缓存、搜索和测试。